

Алгоритмы, модели вычислений и рекурсия

Расширенный конспект

1. Что такое алгоритм и что с ним связано

Алгоритм — это точная, конечная последовательность действий (инструкций), которая переводит вход (входные данные) в выход (результат) за конечное число шагов.

Ключевые слова в этом определении:

- **Точная** — каждый шаг однозначно определён, никакой неопределённости или “на глазок”.
- **Конечная** — алгоритм должен когда-нибудь закончиться (в отличие от, скажем, бесконечного цикла).
- **Вход/выход** — алгоритм не существует “сам по себе”, он всегда что-то получает на вход и что-то выдаёт на выход.

Чтобы говорить об алгоритмах строго (а не на уровне “программа, которая что-то делает”), нужно договориться, на чём алгоритм выполняется и как устроена память, в которой хранятся данные. Именно поэтому понятие алгоритма неразрывно связано с:

- **памятью** — где и как хранятся входные данные, промежуточные результаты и выход;
- **моделью вычислений** (компьютером) — какие операции разрешены и сколько времени (шагов) занимает каждая из них.

Без фиксированной модели вычислений бессмысленно спрашивать “сколько времени работает алгоритм” — секунда на одном устройстве может быть равна тысяче шагов на другом. Поэтому в теории алгоритмов вводят **абстрактные модели вычислений** — упрощённые, но математически строгие описания “компьютера”, на котором алгоритм выполняется. Ниже разбираются две такие модели: машина Тьюринга и RAM-машина.

2. Машина Тьюринга

Машина Тьюринга (MT) — это абстрактная модель вычислительного устройства, состоящая из:

- бесконечной **ленты**, разбитой на ячейки, в каждой из которых хранится один символ из некоторого алфавита;
- **головки**, которая может читать и писать символ в текущей ячейке и сдвигаться на одну ячейку влево или вправо;

- конечного набора **состояний** и **таблицы переходов**, которая по текущему состоянию и прочитанному символу говорит, какой символ записать, куда сдвинуться и в какое состояние перейти.

Несмотря на предельную простоту этой конструкции, машина Тьюринга способна вычислить любую функцию, которую в принципе можно вычислить каким-либо алгоритмом. Это утверждение — не доказанная теорема, а **тезис (гипотеза) Чёрча-Тьюринга**: он не доказывается формально, потому что понятие “вычислимо каким-либо алгоритмом” само по себе интуитивное, а не строго математическое, но за десятилетия все известные модели вычислений (в том числе RAM-машина, которую мы разберём дальше) оказались эквивалентны машине Тьюринга по вычислительной мощности.

Отсюда важный практический вывод, сформулированный в лекции:

Если что-то нельзя посчитать на машине Тьюринга — значит, это нельзя посчитать вообще, никаким алгоритмом и ни на каком устройстве (в том числе на реальном компьютере).

Замечание. Это утверждение — о принципиальной вычислимости, а не об эффективности. Машина Тьюринга может решить задачу, но крайне медленно (например, за экспоненциальное число шагов). Тезис Чёрча-Тьюринга гарантирует, что задача в принципе решается алгоритмически, но ничего не говорит про то, за какое время. Именно поэтому для практических оценок скорости работы алгоритмов удобнее пользоваться не машиной Тьюринга (слишком примитивная модель, неудобно считать реальное время работы), а более приближенной к реальному компьютеру моделью — RAM-машиной.

3. RAM — Random Access Machine

RAM (Random Access Machine, машина с произвольным доступом к памяти) — абстрактная модель вычислений, которая гораздо ближе к реальному компьютеру, чем машина Тьюринга.

Главное отличие от машины Тьюринга: память в RAM-машине представляет собой не ленту (где для доступа к нужной ячейке нужно последовательно сдвигаться), а массив ячеек, к каждой из которых можно обратиться напрямую за один шаг, зная её номер (адрес). Отсюда и название — “произвольный доступ” (random access), в отличие от последовательного доступа на ленте.

word RAM

Уточнение модели RAM, используемое в лекции — **word RAM**: память представляется как массив столбцов (машинных слов) фиксированной длины. То есть память — это не набор отдельных битов или байтов произвольного размера, а набор ячеек одинаковой длины w бит, и за одну операцию машина обрабатывает целиком одно машинное слово (а не бит за битом).

Это соответствует тому, как устроен настоящий процессор: он оперирует не отдельными битами, а словами фиксированной разрядности (например, 32 или 64 бита).

4. Модель вычислений: что считается “одним шагом”

Чтобы можно было измерять время работы алгоритма в “шагах” (а не в реальных секундах, которые зависят от конкретного железа), нужно договориться, какие операции элементарны, то есть выполняются за одно и то же время — за 1 такт.

В модели word RAM принимается соглашение:

Любая операция над двумя машинными словами выполняется за 1 такт (1 шаг).

К таким операциям относятся, например:

- арифметические: $+$, $-$, $*$, $/$, $\%$ (остаток от деления);
- битовые сдвиги: $\gg$ (сдвиг вправо), $\ll$ (сдвиг влево);
- сравнения: $<$, $>$ (а также $=$, $\leq$, $\geq$);
- условные конструкции: `if, else` — переход по условию тоже считается элементарной операцией.

Почему это важно. Такое соглашение — это и есть математическая формализация интуитивного “количества шагов алгоритма”. Именно благодаря ему можно говорить, например, что сложение двух чисел — это $O(1)$ (константное время), а не пытаться считать его в микросекундах, зависящих от процессора.

Важное замечание / где легко ошибиться. Утверждение “операция над двумя словами занимает 1 такт” верно только пока слова имеют фиксированную длину w (то есть пока числа помещаются в одно машинное слово). Если числа настолько велики, что не помещаются в одно слово (например, при работе с “длинной арифметикой”), то операция над ними уже не является элементарной и должна раскладываться на несколько операций над словами — это уже не $O(1)$. Именно с этого наблюдения лекция переходит к следующему пункту — к вопросу о том, что происходит, когда данных (или чисел) становится слишком много.

5. Проблема объёма входных данных и памяти

Возникает естественный вопрос: алгоритм получает входные данные и должен выдать выходные. Но что, если входных данных слишком много, чтобы их вообще можно было записать в память?

Разберём это через устройство памяти в модели word RAM:

- Память состоит из ячеек длиной w бит каждая (машинное слово).
- Одно слово длиной w бит может закодировать ровно 2^w различных значений (каждый бит — 0 или 1, всего независимых комбинаций 2^w).
- Значит, всего адресуемых слов памяти не может быть больше 2^w — ведь если бы слов было больше, то для их нумерации (адресации) не хватило бы возможных значений одного слова.

Отсюда — **закон адресации**:

если размер слова = w бит, то количество адресуемых ячеек памяти $\leq 2^w$.

Смысл этого закона простыми словами: адрес ячейки памяти — это тоже число, и оно тоже должно храниться в одном машинном слове. Раз слово вмещает не больше 2^w различных значений, то и пронумеровать (адресовать) можно не больше 2^w ячеек. Если бы памяти было больше, чем 2^w ячеек, часть из них просто нельзя было бы адресовать — то есть обратиться к ним по номеру не получилось бы.

Практический вывод. Отсюда следует, что размер входных данных, с которыми работает алгоритм в модели word RAM, ограничен разрядностью машинного слова w : если входных элементов больше, чем 2^w , их физически невозможно разместить в адресуемой памяти этой модели. Это и есть формальный ответ на вопрос “что если входных данных слишком много” — модель вычислений сама накладывает это ограничение через величину w .

6. Рекурсия

Рекурсия — это способ решения задачи, при котором алгоритм (функция) вызывает сам себя для решения более мелкой подзадачи того же типа, пока не дойдёт до настолько простого случая (базового случая), что его можно решить напрямую, без дальнейших вызовов.

Рекурсия как дерево

Удобный и наглядный способ анализировать рекурсию — представить её в виде дерева вызовов:

- **корень** дерева — исходный вызов функции с исходными (самыми большими) входными данными;
- **дети** узла — рекурсивные вызовы, которые порождает данный вызов;
- **листья** дерева — вызовы, дошедшие до базового случая (далее не рекурсируют).

Каждый узел дерева соответствует одному вызову функции с определённым размером входных данных.

Оценка рекурсии по времени и по памяти

Дерево рекурсии удобно тем, что через него естественно оцениваются и время работы, и потребление памяти:

- **Время работы** алгоритма соответствует суммарной работе по всем узлам дерева (по всем вызовам): сколько всего было сделано вызовов и сколько “неполезной” работы выполнено в каждом из них (помимо самих рекурсивных вызовов).
- **Память** соответствует глубине (высоте) дерева, а не общему числу узлов. Это важный и часто вызывающий путаницу момент: в каждый конкретный момент времени в памяти “живут” только те вызовы, которые лежат на пути от корня до текущего листа (то есть текущая ветка вызовов), — а не все вызовы дерева сразу. Поэтому потребление памяти рекурсии оценивается через её глубину, то есть через длину самого длинного пути от корня до листа.

Замечание, где легко ошибиться. Не стоит путать “суммарное число узлов дерева” (которое отвечает за оценку времени) и “глубину дерева” (которое отвечает за оценку памяти). Это два разных параметра дерева, и они, вообще говоря, растут по-разному.

7. Применение дерева рекурсии для Merge Sort — подводка к мастер-теореме

Рассмотрим классический пример рекурсивного алгоритма — сортировку слиянием (**Merge Sort**). Алгоритм устроен так:

1. Если массив состоит из одного элемента (или пуст) — он уже отсортирован, это базовый случай.
2. Иначе массив длины n делится на две половины длины $n/2$ каждая.
3. Каждая половина сортируется рекурсивно.
4. Два отсортированных массива сливаются в один за линейное время, то есть за $O(n)$ операций (проход по обоим массивам одновременно).

Запишем время работы алгоритма на входе размера n как рекуррентное соотношение:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n).$$

Смысл этой формулы простыми словами:

- $T(n)$ — время работы на массиве размера n ;
- $2 \cdot T(n/2)$ — время на два рекурсивных вызова для половин массива;
- $O(n)$ — дополнительная (не рекурсивная) работа на текущем уровне рекурсии, то есть слияние двух отсортированных половин.

На этом этапе лекции мастер-теорема ещё не вводится как готовый инструмент — вместо этого предлагается вывести оценку для $T(n)$ самостоятельно, используя дерево рекурсии, а уже затем обобщить полученный приём в виде мастер-теоремы.

Построение дерева рекурсии для Merge Sort

Построим дерево вызовов и посчитаем работу на каждом уровне:

- Уровень 0 (корень): 1 вызов, размер входа n , работа на слияние на этом уровне — $O(n)$.
- Уровень 1: 2 вызова, размер входа у каждого $n/2$, суммарная работа — $2 \cdot O(n/2) = O(n)$.
- Уровень 2: 4 вызова, размер входа у каждого $n/4$, суммарная работа — $4 \cdot O(n/4) = O(n)$.
- ...

- Уровень k : 2^k вызовов, размер входа у каждого $n/2^k$, суммарная работа — $2^k \cdot O(n/2^k) = O(n)$.

Ключевое наблюдение: на каждом уровне дерева суммарная работа одна и та же — $O(n)$, независимо от номера уровня k . Это происходит потому, что при делении задачи пополам число подзадач на уровне растёт ровно в той же пропорции, в которой падает размер каждой подзадачи (умножаем 2^k на $n/2^k$ — и k сокращается).

Осталось понять, сколько всего уровней в этом дереве. Размер задачи на уровне k равен $n/2^k$. Рекурсия доходит до базового случая (размер 1), когда:

$$\frac{n}{2^k} = 1 \implies k = \log_2 n.$$

То есть дерево имеет $\log_2 n$ уровней (это же — глубина дерева, то есть оценка потребления памяти, см. раздел 6).

Так как на каждом из $\log_2 n$ уровней выполняется работа порядка $O(n)$, суммарное время работы всего алгоритма:

$$T(n) = O(n) \cdot \log_2 n = O(n \log n).$$

Это и есть известная оценка сложности Merge Sort — $O(n \log n)$, полученная не “по формуле”, а прямым подсчётом по дереву рекурсии. Именно этот приём — суммирование работы по уровням дерева рекурсии — и лежит в основе мастер-теоремы, к которой лекция переходит далее.

8. Мастер-теорема

Приём, использованный выше для Merge Sort, можно обобщить на любое рекуррентное соотношение вида:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^c),$$

где:

- a — сколько рекурсивных вызовов порождает один вызов (то есть сколько подзадач получается на каждом шаге деления). В Merge Sort $a = 2$ (делим на две половины).
- b — во сколько раз уменьшается размер задачи в каждом рекурсивном вызове. В Merge Sort $b = 2$ (каждая половина в 2 раза меньше исходного массива).
- c — показатель степени, описывающий объём дополнительной (не рекурсивной) работы на каждом вызове — то есть работы, не считая самих рекурсивных вызовов (в Merge Sort это работа по слиянию, $O(n^1)$, то есть $c = 1$).

Мастер-теорема (в том упрощённом виде, который выводится через дерево рекурсии) даёт готовую асимптотическую оценку $T(n)$ через параметры a, b, c , без необходимости каждый раз заново строить и суммировать дерево вручную.

Как выводится мастер-теорема (повторяя приём с деревом)

Построим дерево рекурсии для общего случая $T(n) = a \cdot T(n/b) + O(n^c)$ — точно так же, как это делалось для Merge Sort:

- Уровень 0: 1 вызов, размер n , работа $O(n^c)$.
- Уровень 1: a вызовов, размер каждого n/b , суммарная работа $a \cdot O((n/b)^c)$.
- Уровень 2: a^2 вызовов, размер каждого n/b^2 , суммарная работа $a^2 \cdot O((n/b^2)^c)$.
- Уровень k : a^k вызовов, размер каждого n/b^k , суммарная работа $a^k \cdot O((n/b^k)^c)$.

Заметим, как соотносится работа на уровне k с работой на уровне 0 (это ключевой шаг вывода):

$$\frac{\text{работа на уровне } k}{\text{работа на уровне } 0} = \frac{a^k}{(b^k)^c} = \left(\frac{a}{b^c}\right)^k.$$

То есть от уровня к уровню суммарная работа умножается на множитель $\frac{a}{b^c}$. Именно знак того, больше этот множитель единицы, равен ей или меньше, определяет, какой из уровней “доминирует” в общей сумме — и именно отсюда возникают три случая мастер-теоремы. Удобно ввести обозначение

$$d = \log_b a$$

(это степень, при возведении в которую b даёт a ; можно показать, что условие $a/b^c \geq 1$ равносильно условию $c \leq d$, поскольку $a = b^d$, и тогда $a/b^c = b^{d-c}$, что больше 1 при $d > c$, равно 1 при $d = c$ и меньше 1 при $d < c$).

Три случая мастер-теоремы

Как и в примере с Merge Sort, дерево имеет $k = \log_b n$ уровней (задача уменьшается в b раз на каждом уровне, пока не дойдёт до размера 1). Сравнивая c (степень дополнительной работы на вызов) и $d = \log_b a$ (своего рода “скорость роста числа подзадач”), получаем три случая.

Случай 1: $c < d$ (дополнительная работа растёт медленнее, чем размножаются подзадачи). Тогда множитель $a/b^c > 1$, и работа на каждом следующем уровне больше, чем на предыдущем — сумма по уровням доминируется последним уровнем (листьями дерева). Итоговая оценка:

$$T(n) = \Theta(n^d) = \Theta(n^{\log_b a}).$$

Случай 2: $c = d$ (дополнительная работа растёт наравне с размножением подзадач). Тогда множитель $a/b^c = 1$, и работа одинакова на каждом уровне — это ровно ситуация Merge Sort, разобранный выше. Так как уровней всего $\log_b n$, а работа на каждом порядка n^c , итоговая оценка:

$$T(n) = \Theta(n^c \log n).$$

Случай 3: $c > d$ (дополнительная работа растёт быстрее, чем размножаются подзадачи). Тогда множитель $a/b^c < 1$, и работа на каждом следующем уровне меньше, чем на предыдущем, — сумма по уровням доминируется первым уровнем (корнем дерева, то есть самой “внешней” работой). Итоговая оценка:

$$T(n) = \Theta(n^c).$$

Проверка на примере Merge Sort

Вернёмся к $T(n) = 2 \cdot T(n/2) + O(n)$: здесь $a = 2$, $b = 2$, $c = 1$. Считаем $d = \log_b a = \log_2 2 = 1$. Получаем $c = d$ — это случай 2, и по формуле выше:

$$T(n) = \Theta(n^1 \log n) = \Theta(n \log n),$$

что в точности совпадает с результатом, полученным ранее прямым суммированием по дереву рекурсии. Это подтверждает, что мастер-теорема — не новый факт, а лишь формула, обобщающая приём анализа дерева рекурсии на произвольные a , b , c , чтобы не проделывать суммирование по уровням каждый раз заново вручную.

Важные замечания / где легко ошибиться:

- Параметр c относится к дополнительной (не рекурсивной) работе на *одном вызове*, а не ко всей задаче целиком — легко перепутать эти два понятия.
- $d = \log_b a$ — это не то же самое, что c ; путаница между “числом подзадач/уменьшением размера” (a, b) и “объёмом дополнительной работы” (c) — частая ошибка при применении теоремы.
- Сравнение нужно проводить именно между c и $d = \log_b a$, а не между a и b напрямую.
- Мастер-теорема в этом виде применима только тогда, когда рекурсия честно делит задачу на a подзадач одинакового размера n/b — то есть именно к тому классу рекуррентных соотношений, для которого строилось дерево при выводе.